

Solution to In-Order Traversal (Iterative) / Time and Space Complexity

```
import java.util.Stack;

public class Exercise {
    public void inOrderTraversal(TreeNode root)
    {
        Stack<TreeNode> stack = new Stack<>();
        TreeNode current = root;

        while (current != null ||
!stack.isEmpty()) {
            while (current != null) {
                stack.push(current);
                current = current.left;
            }
            current = stack.pop();
            System.out.print(current.val + "
");
            current = current.right;
        }
    }
}
```

Code Breakdown

1. **import java.util.Stack;**
 - Imports the **Stack** class from the **java.util** package, which will be used to store **TreeNode** objects.
2. **public class Exercise {**
 - Declaration of the **Exercise** class.
3. **public void inOrderTraversal(TreeNode root) {**
 - Starts the definition of the **inOrderTraversal** method, which takes the root of a binary tree as an argument.
4. **Stack<TreeNode> stack = new Stack<>();**

- Creates a new stack that will store **TreeNode** objects. This stack is used to manually manage the traversal process.
5. **TreeNode current = root;**
 - Initializes a **TreeNode** reference **current** and sets it to the root of the tree. This variable will be used to traverse the tree.
 6. **while (current != null || !stack.isEmpty()) {**
 - Begins a **while** loop that continues as long as **current** is not null or the stack is not empty. This condition ensures that all nodes are visited.
 7. **while (current != null) {**
 - Starts an inner loop to explore the leftmost node of the current subtree.
 8. **stack.push(current) ;**
 - Pushes the current node onto the stack. This action saves the current node so that we can return to it after exploring its left subtree.
 9. **current = current.left;**
 - Moves the **current** reference to its left child, going deeper into the left subtree.
 10. **}**
 - Ends the inner while loop.
 11. **current = stack.pop() ;**
 - Once the leftmost node is reached (i.e., **current** is null), this line pops the top element from the stack. This node is the next in in-order sequence.
 12. **System.out.print(current.val + " ");**
 - Prints the value of the current node. In in-order traversal, the node is processed after its left subtree and before its right subtree.
 13. **current = current.right;**

- Moves the **current** reference to its right child, preparing to explore the right subtree in the next iterations of the loop.
14. **}**
 - Ends the outer while loop.
 15. **}**
 - Ends the **inOrderTraversal** method.
 16. **}**
 - Ends the **Exercise** class.

Time Complexity

- **O(n)**: Each node in the binary tree is visited exactly once. The in-order traversal visits every node in the tree, and each node is pushed to and popped from the stack exactly once.

Space Complexity

- **O(h)**: The space complexity is determined by the height of the tree, **h**, because, in the worst case, the stack can contain all the nodes on one branch of the tree (from the root to the leaf). For a balanced tree, **h** is $\log(n)$, and for a skewed tree (worst case), **h** can be **n**.

This in-order traversal method is an efficient way to traverse a binary tree iteratively, especially when recursion is not desired or feasible.

Solution to Pre-Order Traversal (Iterative) / Time and Space Complexity

```
import java.util.Stack;

public class Exercise {
    public void preOrderTraversal(TreeNode
root) {
        if (root == null) return;
```

```

Stack<TreeNode> stack = new Stack<>();
stack.push(root);

while (!stack.isEmpty()) {
    TreeNode node = stack.pop();
    System.out.print(node.value + " ");

    if (node.right != null) {
        stack.push(node.right);
    }
    if (node.left != null) {
        stack.push(node.left);
    }
}
}
}

```

Code Breakdown

1. **import java.util.Stack;**
 - Imports the **Stack** class from the **java.util** package for use in the program. Stacks are used to hold references to tree nodes.
2. **public class Exercise {**
 - Begins the definition of the **Exercise** class.
3. **public void preOrderTraversal(TreeNode root) {**
 - Starts the definition of the **preOrderTraversal** method. This method takes the root node of a binary tree as an argument.
4. **if (root == null) return;**
 - Checks if the root node is null. If it is, the method returns immediately as there's nothing to traverse.
5. **Stack<TreeNode> stack = new Stack<>();**

- Creates a new instance of **Stack** that will hold **TreeNode** objects. This stack is used to manage the traversal process iteratively.
6. **stack.push(root) ;**
 - Pushes the root node onto the stack. This is the starting point of the traversal.
 7. **while (!stack.isEmpty()) {**
 - Begins a **while** loop that continues as long as the stack is not empty. This ensures that all nodes are visited.
 8. **TreeNode node = stack.pop() ;**
 - Pops the top node from the stack. This node will be processed and its children will be examined.
 9. **System.out.print(node.value + " ");**
 - Prints the value of the current node. In pre-order traversal, the node is processed before its children.
 10. **if (node.right != null) {**
 - Checks if the right child of the current node is not null.
 11. **stack.push(node.right) ;**
 - If the right child exists, it is pushed onto the stack.
 12. **if (node.left != null) {**
 - Checks if the left child of the current node is not null.
 13. **stack.push(node.left) ;**
 - If the left child exists, it is pushed onto the stack. Note that the left child is pushed after the right child so that it is processed first, maintaining the pre-order sequence.
 14. **}**
 - Ends the **while** loop.
 15. **}**
 - Ends the **preOrderTraversal** method.
 16. **}**
 - Ends the **Exercise** class.

Time Complexity

- **$O(n)$** : Each node in the binary tree is visited exactly once. Since the traversal visits every node in the tree, and operations for each node (pushing to and popping from the stack) are constant-time operations, the overall time complexity is $O(n)$, where n is the number of nodes in the tree.

Space Complexity

- **$O(h)$** : The space complexity is determined by the height of the tree, h , because, in the worst case, the stack can contain all the nodes on one branch of the tree (from the root to a leaf). For a balanced tree, h is $\log(n)$, and for a skewed tree (worst case), h can be n .

The iterative pre-order traversal method is efficient and particularly useful in situations where recursion is not feasible or when the call stack size is a concern.

Solution to Post-Order Traversal (Iterative) / Time and Space Complexity

```
import java.util.Stack;

public class Exercise {
    public void postOrderTraversal(TreeNode
root) {
        Stack<TreeNode> stack1 = new Stack<>();
        Stack<TreeNode> stack2 = new Stack<>();

        if (root != null) stack1.push(root);

        while (!stack1.isEmpty()) {
            TreeNode node = stack1.pop();
            stack2.push(node);
        }
    }
}
```

```

        if (node.left != null)
stack1.push(node.left);
        if (node.right != null)
stack1.push(node.right);
    }

    while (!stack2.isEmpty()) {
        TreeNode node = stack2.pop();
        System.out.print(node.value + " ");
    }
}
}

```

Code Explanation

1. **import java.util.Stack;**
 - This line imports the **Stack** class from the **java.util** package, which is used to create stack data structures in the program.
2. **public class Exercise {**
 - Begins the definition of a class named **Exercise**.
3. **public void postOrderTraversal(TreeNode root)**
{
 - Declares the **postOrderTraversal** method with a parameter **root** of type **TreeNode**. This method will perform the post-order traversal on a binary tree.
4. **Stack<TreeNode> stack1 = new Stack<>();**
 - Creates a **Stack** object named **stack1** to hold **TreeNode** objects. This stack is used to perform the initial traversal of the tree.
5. **Stack<TreeNode> stack2 = new Stack<>();**

- Creates another **Stack** object named **stack2**. This stack is used to reverse the order of nodes to achieve the post-order traversal.
6. **if (root != null) stack1.push(root);**
 - If the root is not null, it is pushed onto **stack1**. This check prevents adding a null root to the stack.
 7. **while (!stack1.isEmpty()) {**
 - Starts a **while** loop that runs as long as **stack1** is not empty.
 8. **TreeNode node = stack1.pop();**
 - Pops the top element from **stack1** and assigns it to **node**. This node will be processed in this iteration.
 9. **stack2.push(node);**
 - Pushes the popped node onto **stack2**. This is done to reverse the order later.
 10. **if (node.left != null)**
stack1.push(node.left);
 - If the left child of **node** is not null, it is pushed onto **stack1**. This is done to ensure that the left child is processed before the right child.
 11. **if (node.right != null)**
stack1.push(node.right);
 - If the right child of **node** is not null, it is pushed onto **stack1**.
 12. **}**
 - Ends the **while** loop for **stack1**.
 13. **while (!stack2.isEmpty()) {**
 - Starts another **while** loop that runs as long as **stack2** is not empty.
 14. **TreeNode node = stack2.pop();**

- Pops the top element from `stack2` and assigns it to `node`. Nodes are popped in the reverse order of their initial visitation.
15. `System.out.print(node.value + " ");`
 - Prints the value of `node`. Since `stack2` reverses the order of visitation, this results in a post-order traversal output.
 16. `}`
 - Ends the `while` loop for `stack2`.
 17. `}`
 - Ends the `postOrderTraversal` method.
 18. `}`
 - Ends the `Exercise` class.

Time Complexity

- **O(n)**: Each node in the binary tree is visited exactly once and each visit involves constant time operations (pushing and popping on stacks). Here, `n` is the number of nodes in the binary tree.

Space Complexity

- **O(n)**: In the worst case, each of the two stacks could hold up to `n` nodes at the same time (particularly when the tree is skewed). Therefore, the space complexity is O(n).

This method is particularly useful for iterative post-order traversal as it avoids recursion, which can be beneficial in environments where stack size (for recursion) is limited.